다수이종센서융합SW설계

Sensor Fusion

청주대학교 무인항공기학과
김상현 교수 (www.intro-lab.com)

디폴트 인수

- 인수를 전달하지 않아도 디폴트값을 대신 넣어주는 기능을 디폴트 인수 (default argument)라고 한다.

```
#include <iostream>
using namespace std;

// 문자 c를 n번 반복하여 화면에 출력한다.
void display(char c = '*', int n = 10)
{
        for (int i = 0; i < n; i++)
                cout << c;
        cout << endl;
}
```

디폴트 인수

```
#include <iostream>
using namespace std;

void display(char c = '*', int n = 10)
{
        for (int i = 0; i < n; i++)
                cout << c;
        cout << endl;
}
int main()
{
        cout << "아무런 인수가 전달되지 않는 경우:\n";
        display();

        cout << "\n첫 번째 인수만 전달되는 경우:\n";
        display('#');

        cout << "\n모든 인수가 전달되는 경우:\n";
        display('#', 5);

        return 0;
}
```

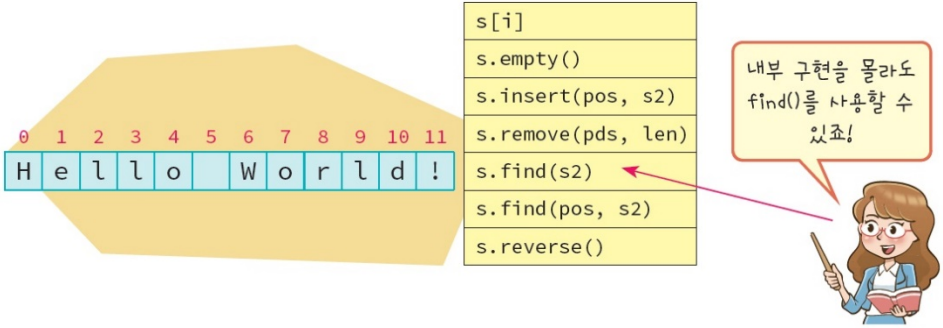
인라인 함수

- 함수 이름 앞에 inline이 붙으면 컴파일러는 함수를 생성하지 않고 함수의 코드를 호출한 곳에 직접 집어넣는다.

```
// 실수값을 제곱하는 함수
inline double square(double i)
{
        return i*i;
}
```

문자열

◆ string 클래스



◆ 클래스와 객체

string 클래스 사용하기

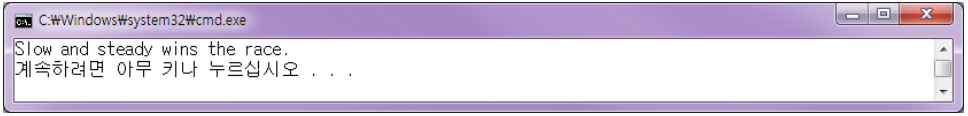
```
#include <string>
using namespace std;

void main()
{

        string s;                          // string 객체 s를 생성한다.
        string s = "Hello World!";         // string 객체를 생성하고 초기화한다.
        string s{ "Hello World!" };        // string 객체를 생성하고 초기화한다.
}
```

문자열의 결합

```
#include <string>
using namespace std;

void main()
{

string subject = "Money";
string other = " has no value if it is not used";
string sentence = subject + other;
}
```

예제

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
        string s1 = "Slow", s2 = "steady";
        string s3 = "the race.";
        string s4;

        s4 = s1 + " and " + s2 + " wins " + s3;
        cout << s4 << endl;
        return 0;
}
```



문자열의 비교

```
#include <string>
using namespace std;

void main()
{
        string s1 = "Hello", s2 = "World";
        if( s1 == s2 )
                cout << "동일한 문자열입니다." << endl;
        else
                cout << "동일한 문자열이 아닙니다." << endl;
        if( s1 > s2 )
                cout << "s1이 앞이 있습니다. " << endl;
        else
                cout << "s2가 앞이 있습니다. " << endl;

}
```

string 클래스 멤버 함수 사용

멤버 함수	설명
s[i]	i번째 원소
s.empty()	s가 비어있으면 true 반환
s.insert(pos, s2)	s의 pos 위치에 s2를 삽입
s.remove(pos, len)	s의 pos 위치에 len만큼을 삭제
s.find(s2)	s에서 문자열 s2가 발견되는 첫번째 인덱스를 반환
s.find(pos, s2)	s의 pos 위치부터 문자열 s2가 발견되는 첫번째 인덱스를 반환

예제

◆ 문자열 "When in Rome, do as the Romans."중에서 "Rome"이 몇 번째 위치에 있는지를 계산하는 프로그램

```
C:₩Windows₩system32₩cmd.exe
8
계속하려면 아무 키나 누르십시오 . . .
```

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
        string s="When in Rome, do as the Romans.";

        int index = s.find("Rome");
        cout << index << endl;
        return 0;
}
```

예제

◆ 사용자가 입력한 주민등록번호에서 '-' 문자를 삭제하는 프로그램을 작성하여 보자.

```
C:₩Windows₩system32₩cmd.exe
주민등록번호를 입력하시오: 123456-123456
-가 제거된 주민등록번호: 123456123456
계속하려면 아무 키나 누르십시오 . . .
```

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
        string s;
        cout << "주민등록번호를 입력하시오: ";
        cin >> s;

        cout << "-가 제거된 주민등록번호: ";
        for (auto& c : s) {
                if (c == '-') continue;
                cout << c;
        }
        cout << endl;

        return 0;
}
```

문자열의 배열

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
	string list[] = { "철수", "영희", "길동" };

	for (auto& x : list)
		cout << (x + "야 안녕!") << endl;
	return 0;
}
```

객체지향

◆ **객체 지향 프로그래밍(OOP: object-oriented programming)**은 우리가 살고 있는 실제 세계가 객체(object)들로 구성되어 있는 것과 비슷하게, 소프트웨어도 객체로 구성하는 방법이다.

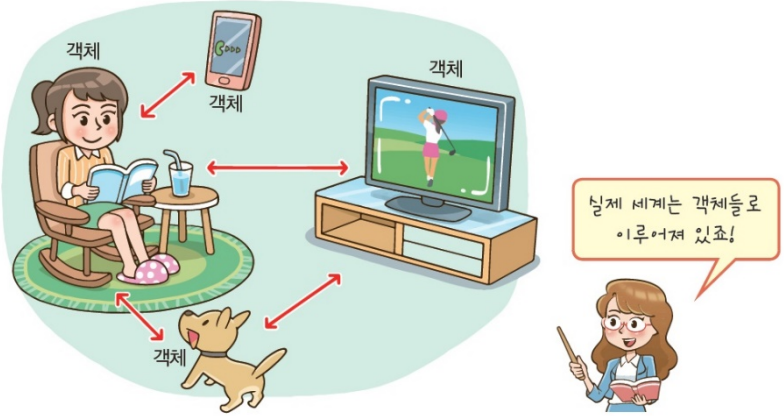


그림 4.1 실제 세계는 객체들로 이루어진다.

객체와 메시지

◆ 객체들은 메시지를 주고 받으면서 상호작용한다.

그림 4.2 객체들은 서로 메시지를 주고받으면서 상호작용한다.

절차 지향과 객체 지향

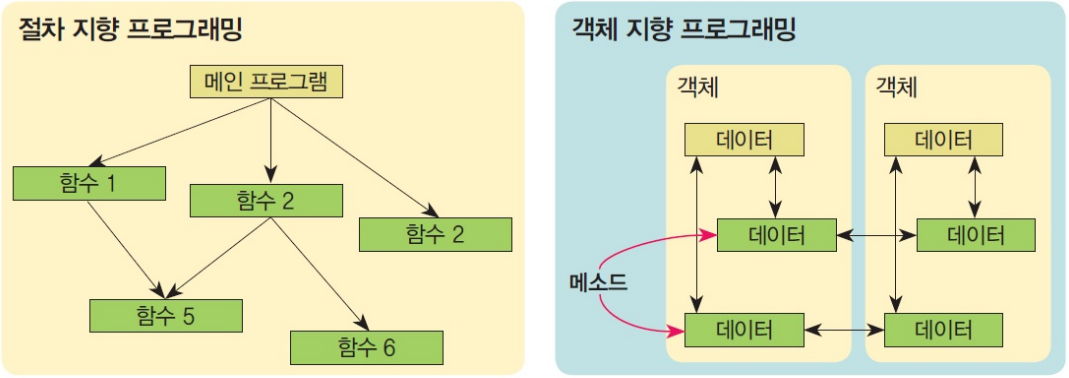
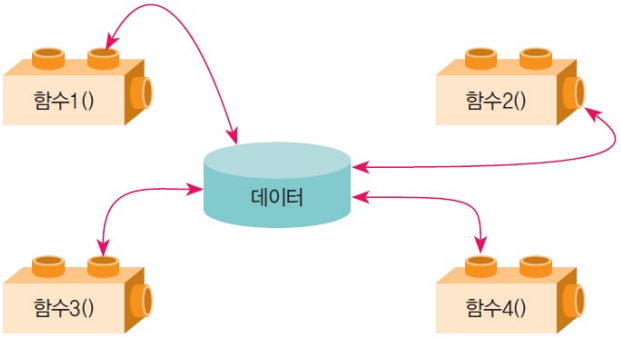


그림 4.3 절차 지향 프로그래밍과 객체 지향 프로그래밍의 비교

절차지향

- **절차 지향 프로그래밍(procedural programming)**은 **프로시저(procedure)**를 기반으로 하는 프로그래밍 방법이다.
- 프로시져는 일반적으로 함수를 의미한다.
- 절차 지향 프로그래밍에서 전체 프로그램은 함수들의 집합으로 이루어진다.

절차지향의 문제점



절차 지향 프로그래밍에서는 데이터와 함수가 묶여 있지 않다.

그림 4.4 절차 지향 프로그래밍

객체지향 프로그래밍

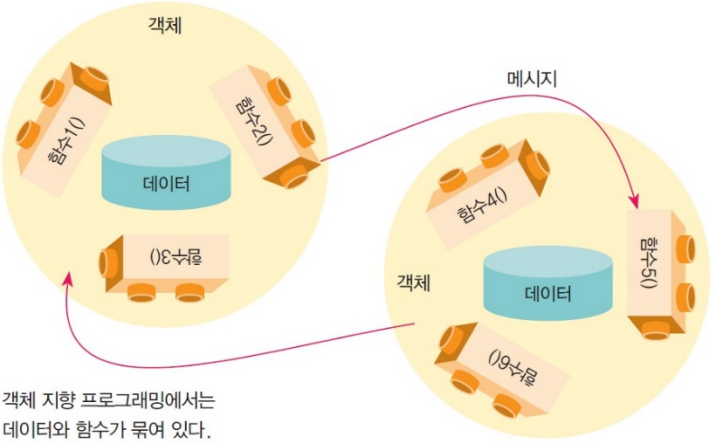


그림 4.5 객체 지향 프로그래밍

객체의 구성

◆ 객체는 상태와 동작을 가지고 있다. **객체의 상태(state)**는 객체의 속성이다. **객체의 동작(behavior)**은 객체가 취할 수 있는 동작이다.

그림 4.7 자동차 객체의 예

멤버 변수와 멤버 함수

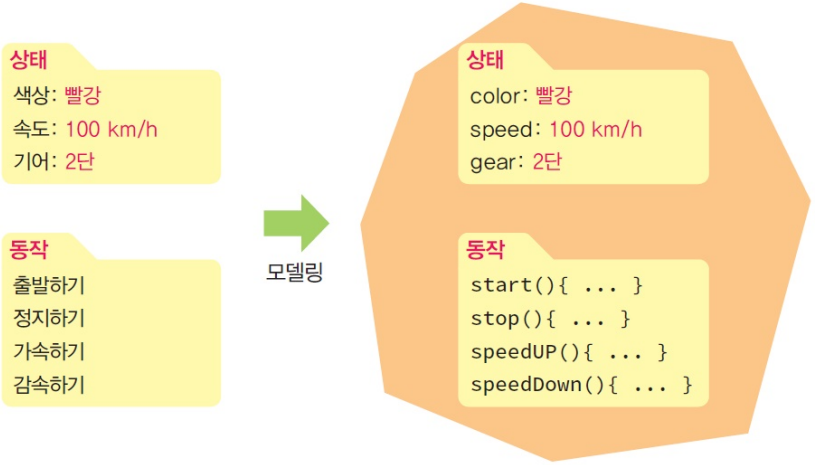


그림 4.8 멤버 변수와 멤버 함수

클래스 = 객체의 설계도

◆ 객체 지향 소프트웨어에서도 같은 객체들이 여러 개 필요한 경우도 있다. 이러한 객체들은 모두 하나의 설계도로 만들어진다. 바로 이 설계도를 클래스(class)라고 한다.

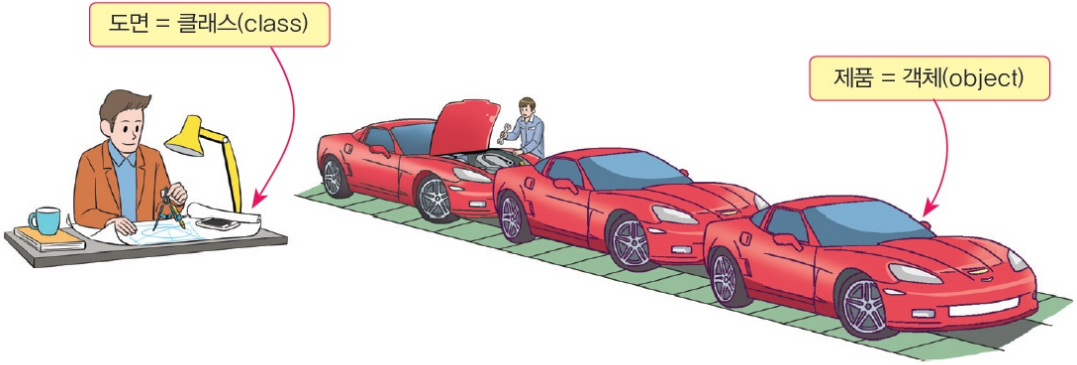


그림 4.9 객체를 클래스라는 설계도로 생성된다.

클래스 작성하기

문법 5.1 클래스 정의

```
class 클래스이름 {
    자료형 멤버변수1;
    자료형 멤버변수2;

    반환형 멤버함수1();
    반환형 멤버함수2();
};
```

멤버 변수

멤버 함수 선언부

클래스 작성의 예

class 키워드로 클래스 선언

클래스 이름

접근 지정자

멤버 변수

멤버 함수

```
class Circle {
public:
    int radius;
    string color;

    double calcArea() {
        return 3.14*radius*radius;
    }
};
```

접근지정자

- private 멤버는 클래스 안에서만 접근(사용)될 수 있다.
- protected 멤버는 클래스 안과 상속된 클래스에서 접근이 가능하다(상속은 아직 학습하지 않았다).
- public 멤버는 어디서나 접근이 가능하다.

객체 생성하기

```
Circle obj;          // obj는 Circle 자료형의 변수이다.
```

클래스 이름은 자료형의 이름으로 생각할 수 있다.

객체의 이름

클래스는 객체를 찍어내는 틀과 같다.

객체의 멤버 접근

◆ 멤버에 접근하기 위해서는 도트(.) 연산자를 사용한다.

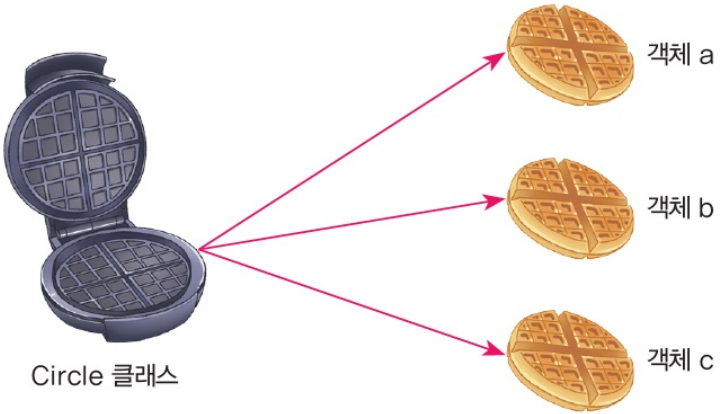
```
obj.radius = 3;            // obj의 멤버 변수인 radius에 3을 저장한다.
```

obj 객체의

radius 멤버 변수에 접근

```
#include <iostream>
using namespace std;

class Circle {
public:
        int radius;             // 반지름
        string color;           // 색상

        double calcArea() {
                return 3.14*radius*radius;
        }
};

int main() {
        Circle obj;

        obj.radius = 100;
        obj.color = "blue";

        cout << "원의 면적=" << obj.calcArea() << "\n";
        return 0;
}
```

하나의 클래스로 많은 객체 생성 가능



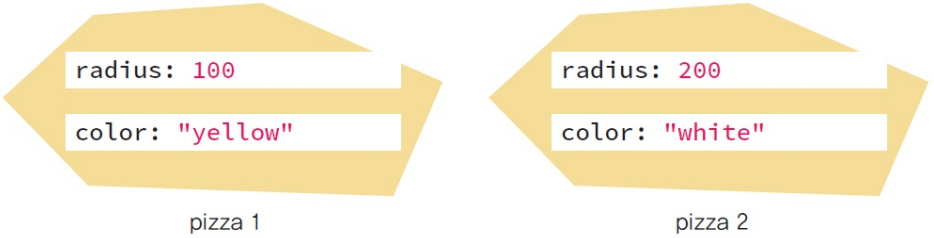
하나의 클래스로 많은 객체 생성 가능

```
int main()
{
        Circle pizza1, pizza2;

        pizza1.radius = 100;
        pizza1.color = "yellow";
        cout << "피자의 면적=" << pizza1.calcArea() << "\n";

        pizza2.radius = 200;
        pizza2.color = "white";
        cout << "피자의 면적=" << pizza2.calcArea() << "\n";
        return 0;
}
```

```
C:₩Windows₩system32₩cmd.exe
피자의 면적=31400
피자의 면적=125600
계속하려면 아무 키나 누르십시오 . . .
```

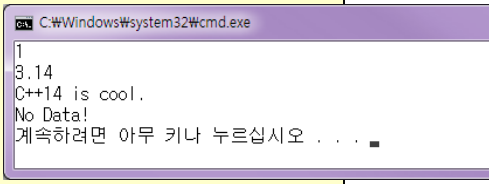
각 객체의 상태

◆ 각 객체의 멤버 변수 값은 서로 다르다.



멤버 함수 중복 정의

```
#include <iostream>
#include <string>
using namespace std;

class PrintData {
public:
        void print(int i) { cout << i << endl; }
        void print(double  f) { cout << f << endl; }
        void print(string s = "No Data!") { cout << s << endl; }
};

int main() {
        PrintData obj;

        obj.print(1);
        obj.print(3.14);
        obj.print("C++14 is cool.");
        obj.print();
        return 0;
}
```



클래스의 인터페이스와 구현의 분리

◆ 복잡한 클래스인 경우에는 멤버 함수를 클래스 외부에서 정의

```
#include <iostream>
using namespace std;

class Circle {
public:
        double calcArea();

        int radius;         // 반지름
        string color;       // 색상
};
```

```
// 클래스 외부에서 멤버 함수들이 정의된다.
double Circle::calcArea() {
        return 3.14*radius*radius;
}

int main()
{
        Circle c;
        c.radius = 10;
        cout << c.calcArea() << endl;
        return 0;
}
```

C:₩Windows₩system32₩cmd.exe

```
314
계속하려면 아무 키나 누르십시오 . . .
```

이름 공간

- 이름 공간(name space)는 식별자 (자료형, 함수, 변수 등의 이름)의 영역
- 이름 공간은 코드를 논리적 그룹으로 구성하고 특히 코드에 여러 라이브러리가 포함되어 있을 때 발생할 수 있는 이름 충돌을 방지하는 데 사용된다.

```
using namespace std;
```

using 문장을 사용하지 않으면

```
#include <iostream>

class Circle {
public:
        double calcArea();

        int radius;        // 반지름
        std::string color;        // 색상
};
double Circle::calcArea() {
        return 3.14*radius*radius;
}

int main() {
        Circle c;
        c.radius = 10;
        std::cout << c.calcArea() << std::endl;
        return 0;
}
```

클래스의 선언과 클래스의 정의 분리

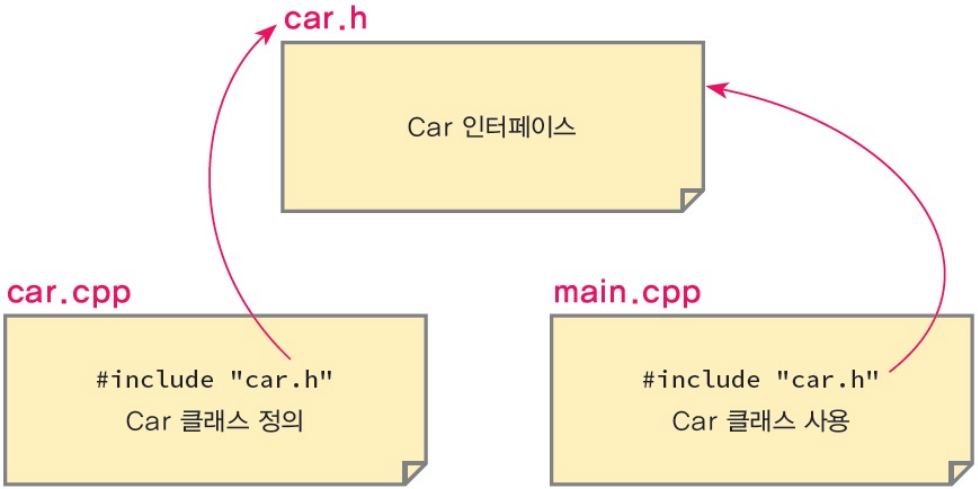


그림 4.11 클래스를 헤더 파일과 소스 파일로 분리

클래스의 선언과 클래스의 정의 분리

◆ car.h

```
#include <iostream>
#include <string>
using namespace std;

class Car
{
        int speed;      //속도
        int gear;               //기어
        string color;   //색상
public:
        int getSpeed();
        void setSpeed(int s);
};
```

클래스의 선언과 클래스의 정의 분리

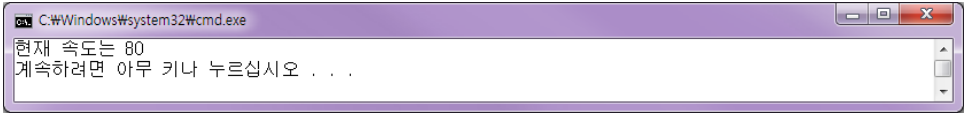
◆ car.cpp

```
#include "car.h"

int Car::getSpeed()
{
        return speed;
}
void Car::setSpeed(int s)
{
        speed = s;
}
```

클래스의 선언과 클래스의 정의 분리

◆ main.cpp

```
#include "car.h"
using namespace std;

int main()
{
        Car myCar;

        myCar.setSpeed(80);
        cout << "현재 속도는 " << myCar.getSpeed() << endl;

        return 0;
}
```



Q&A